Learning Objectives

- Use Git to keep track of the development of a software project
- Use a Makefile to automate compilation and linking
- Extend the grammar with additional binary operators and precedence levels
- Recognize keywords in the lexer

Step 1 — Putting Our Compiler Project under Version Control

If you are already using Git for your compiler project, you may skip this step — or, even better, help someone who is using Git for the first time.

At this point, your `not-abc` directory should contain all the source and header files that we have created for our compiler project. It may also contain various other files that accumulated while experimenting: object files, executables, generated `.tex` and `.pdf` files, and perhaps a few other things.

Our compiler has grown enough that it would be a good idea to secure the files that actually matter — in particular our `.abc` and `.hdr` files. We could, of course, simply make a backup copy of them somewhere.

For a software project, however, we usually want more than just a backup. We would like to:

- keep a history of meaningful versions of our project,
- record what changed between these versions,
- record why we made a particular change,
- return to an earlier working version if an experiment goes wrong,
- and distinguish files that belong to the project from files that can easily be generated again.

This is exactly what a version control system is for. We will use Git, which lets us record useful states of our project as commits and return to them later if necessary.

Git can do much more, but we will not try to learn everything at once. Instead, we will introduce additional Git features throughout this and later worksheets — or when a good opportunity comes up in class. This way, we learn them when we actually have a problem they can solve, rather than learning solutions to problems we have never encountered.

Getting Our Project into a Git Repository

Setting up an account on a hosting service such as GitHub or GitLab, creating SSH keys, and configuring access involves several service-specific steps that may change over time. These are therefore described in a separate document.
For the following tasks, we assume that you have created a new empty repository `not-abc` and can access it from the command line. Make sure you do not ignore any error messages in the following:

1. Clone the empty `not-abc` repository into a suitable directory (e.g. `~/hpc0/session15`):

   ```
   cd ~/hpc0/session15
   git clone <URL-of-your-repository>
   ```

   Make sure that you use the SSH URL of your repository (typically something like `git@github.com:...`) and not the HTTPS URL. We want to push changes back to the repository later, using the SSH authentication that you configured before. Git will probably warn you that you have cloned an empty repository. That is exactly what we expect.
2. Your old `not-abc` directory contains the compiler project that we now want to put under version control. Copy only the `.abc` and `.hdr` files from there into the newly cloned `not-abc` directory. Fortunately, by now you have enough experience with the terminal and the shell to figure out how to do this. ;-)
   Change into the new directory and carefully check its contents before continuing. It should contain the source and header files of your compiler project, but not object files, executables, PDFs, or other files produced while experimenting.
3. Tell Git which files should become part of the next commit:

   ```
   git add *.abc *.hdr
   ```

   Specifying the patterns explicitly gives us another small safeguard in case some generated files accidentally found their way into the directory.

© Michael C. Lehn | CC BY-SA 4.0

4. Check what Git is about to record:

   ```
   git status
   ```

   Take a careful look at the output. If you are not sure what Git is telling you, **ask us** — or ask ChatGPT (or your favorite LLM) to explain the output. Make sure you understand it before continuing with the commit.
5. If everything looks right, create your first commit:

   ```
   git commit -m "Add compiler project"
   ```

   `git add` selects changes for the next commit. `git commit` records these changes in the local history of the repository.
   Finally, send the new commit to the repository on GitHub, GitLab, or whichever hosting service you are using:

   ```
   git push
   ```

   In other words: add selects, commit records locally, and push sends commits to the remote repository.
6. Open your repository in a web browser and check that your `.abc` and `.hdr` files are actually there.
7. **The moment of truth.** If everything looks correct in the browser, delete all your local not-abc directories — including the repository you just cloned. At this point, the only remaining copy of your compiler project should be the remote repository.
   1. If this makes you nervous, **ask us to stand next to you** while you do it. ;-)
   2. Now clone the repository again and verify that all your source and header files reappear.
   3. If something went wrong in the previous tasks, quite a bit of typing may be waiting for you. ;-)

Step 2 — Two Useful Command-Line Tools: `curl` and `wget`

If you followed the HPC0 setup instructions from Session 0, both `curl` and `wget` should be installed on your system. Both are command-line tools for downloading files from a URL.
These are useful tools to have used at least once. You do not need to memorize their many options. Being able to quickly figure out how to use a tool for a particular task — even if you have forgotten the details 30 minutes later — is part of everyday work on a Unix-like system.
We have prepared a `Makefile` for our compiler project in the `hpc0-stuff` repository

`https://github.com/michael-lehn/hpc0-stuff`

1. Change into your `not-abc` directory. On GitHub, navigate to the Makefile and find the URL for the raw file.
2. Use `curl` to download the `Makefile`. The basic form is `curl [options] <URL>`. By default, `curl` writes the downloaded data to standard output. Option `-o` can be used to specify a file instead: `curl -o <filename> <URL>`. For our `Makefile`, use[1]:

   ```
   curl -o Makefile <raw-URL-of-the-Makefile>
   ```

   If a file called `Makefile` already exists, it will be **overwritten** — which is exactly what we want here. Check that the file is actually there and take a quick look at its contents.

   `curl` has many more options. If you are curious, have a quick look at `curl –help` or, old-school, `man curl`. You do not need to memorize them. The important thing is to know how to find out what you need when you need it.
3. Now let us do the same thing with `wget`. First remove the Makefile downloaded with `curl`:

   ```
   rm Makefile
   ```

   Download it again and use ls to check that it is back:

   ```
   wget <raw-URL-of-the-Makefile>
   ls
   ```

   Now run the same `wget` command once more and check again with `ls`. You should notice that `wget` does not overwrite the existing file but creates another one instead. If we want to

[1] On GitHub, you can easily get the raw URL of a file through the web interface — ask us if you are not sure where to find it. Then good old copy & paste saves you from typing the whole thing. ;-) For our Makefile, the raw URL is `https://raw.githubusercontent.com/michael-lehn/hpc0-stuff/refs/heads/main/Makefile`.

© Michael C. Lehn | CC BY-SA 4.0

overwrite Makefile, we can explicitly specify the output file with option `-O`:

```
wget -O Makefile <raw-URL-of-the-Makefile>
```

As with `curl`, wget has many more options. Use `wget --help` or, old-school, `man wget` if you need them.

4. Finally, add the new Makefile to your own repository, commit the change, and push it:

```
git add Makefile
git commit -m "Add Makefile"
git push
```

Check your repository in the browser again. The Makefile should now be there as well.

Step 3 — Let `make` Do the Work

Our compiler project already consists of quite a few files. We currently have several test programs,

`xtest_expr.abc` `xtest_lexer.abc` `xtest_parseexpr.abc` `xtest_symtab.abc` `xtest_ustr.abc`

the corresponding implementation files,

`expr.abc` `lexer.abc` `parseexpr.abc` `symtab.abc` `ustr.abc`

and their header files,

`expr.hdr` `lexer.hdr` `parseexpr.hdr` `symtab.hdr` `ustr.hdr`

Compiling and linking all test programs manually is already becoming rather tedious. Fortunately, the `Makefile` that you downloaded in the previous step can automate this for us.

1. Run

   ```
   make
   ```

   and watch carefully what happens. Afterwards, use ls to inspect your project directory. Among other things, you should now find executable test programs such as

   `xtest_expr` `xtest_lexer` `xtest_parseexpr` `xtest_symtab` `xtest_ustr`

2. You should also find a new directory called `obj`. Have a look inside:

   ```
   ls obj
   ```

   For every `.abc` source file that was compiled, you should find a corresponding object file in this directory. Recall the video More on Translation Units: these object files contain the separately compiled translation units that are later linked to form the executable programs.

3. There is another new directory called `dep`. Have a look:

   ```
   ls dep
   ```

   It contains a dependency file for each source file. For example, take a look at

   ```
   cat dep/expr.abc.d
   ```

   You should see something similar to

   ```
   obj/expr.o: expr.abc expr.hdr ustr.hdr /usr/local/include/abc/stdabc.hdr ...
   expr.hdr:
   ustr.hdr:
   ...
   ```

   You do not need to understand the details of this file. One important piece of information, however, should already make sense: it tells make that `obj/expr.o` depends on files including `expr.abc`, `expr.hdr`, and `ustr.hdr`.

   Consequently, if one of these files changes, `obj/expr.o` may no longer correspond to the current source code and has to be compiled again.

Now Let Us Experiment

4. Run `make` once more without changing anything. What happens?
5. Now pretend that you edited `expr.abc`:

   ```
   touch expr.abc
   make
   ```

   Watch carefully which commands are executed. Which object file is rebuilt? Which test programs are relinked?

© Michael C. Lehn | CC BY-SA 4.0

6. Repeat the experiment with a header file:

   ```
   touch ustr.hdr
   make
   ```

   This time, more than one source file may be affected. Compare what make does with the dependency files in `dep`.
7. Now, try

   ```
   make clean
   ls
   ```

   What disappeared? Now run `make` again. Compare what happens with the previous incremental builds.
8. Finally, try

   ```
   touch lexer.abc
   make
   ```

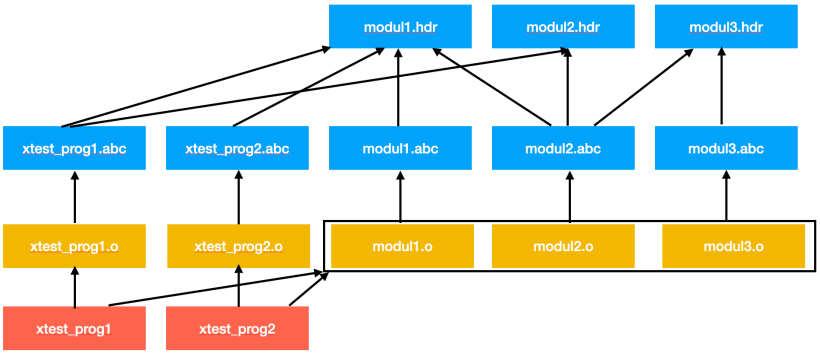
   Why can changing a single .abc file cause several test programs to be relinked?

A bit more insight …

Let us put together what we have just observed.

`make` uses the dependencies between files together with their modification times to decide what needs to be rebuilt. If none of the files on which a target depends has changed, there is nothing to do. If a source or header file changes, only the affected object files need to be recompiled. Executables depending on these object files then need to be relinked.

This is called an incremental build: instead of rebuilding the complete project every time, we rebuild only what may have become outdated.



A bit more insight …

So far, our `Makefile` has saved us from manually compiling and linking all the different parts of our compiler project. Now it is time to make some actual changes — and we will soon be glad that we have it. ;-)
At the end of Worksheet 13, our parser implemented a grammar equivalent to

```
expr                = additive-expr
additive-expr       = multiplicative-expr { "+" multiplicative-expr }
multiplicative-expr = factor { "*" factor }
factor              = decimal-literal
                    | "(" expr ")"
```

We eventually replaced the separate parsing functions for additive-expr and multiplicative-expr by the more general

```
fn parseBinary(prec: int): ->Expr
```

where `parseBinary(12)` basically plays the role of `additive-expr`, and `parseBinary(13)` plays the role of `multiplicative-expr`.
The next step gives us a good opportunity to revisit how this works. We will extend the grammar — which will require changes in several source and header files. Fortunately, from now on a simple make will take care of figuring out what needs to be rebuilt.

© Michael C. Lehn | CC BY-SA 4.0

Step 4 — Extending the Grammar

We want our compiler to support a few more binary operators. Extend the grammar to

```
expr                = additive-expr
additive-expr       = multiplicative-expr { ("+" | "-") multiplicative-expr }
multiplicative-expr = factor { ("*" | "/" | "%") factor }
factor              = decimal-literal
                    | "(" expr ")"
```

Notice that we are not introducing any new precedence levels. `+` and `-` have the same precedence, while `*`, `/`, and `%` have a higher precedence.

There are several ways we could approach this extension. Since we have just spent some time thinking about dependencies, however, let us use a slightly different strategy here. We will deliberately make the right change at one place first and then let the consequences guide us through the rest of the compiler. Two things can happen:

- the compiler tells us that something is still missing, or
- one of our tests reveals that we have not yet implemented everything consistently.

In other words, instead of making a list of all files that need to be changed beforehand, we will knock over the first domino and see where it leads us. ;-)

The first domino will be function `tokenKindPrec` in `parseexpr.abc`. Change it to

```
fn tokenKindPrec(kind: TokenKind): int
{
    switch (kind) {
        default: return 0;
        case ASTERISK: return 13;
        case SLASH: return 13;
        case PERCENT: return 13;
        case PLUS: return 12;
        case MINUS: return 12;
    }
}
```

If you understood how parseBinary works, convince yourself that with this change our parser now implements the extended grammar shown above.

Tasks

1. Compile everything with `make`. Everything should compile without errors.

   Now run `./xtest_parseexpr` and enter `2 - 1` (followed by `Ctrl-D`) This time we get a runtime error:

   ```
   parseexpr.abc:19: Assertion '0 && "internal error: exprKindFromToken"' failed.
   ```

   As the error message tells us, the assertion was triggered in `parseexpr.abc`, line 19, inside function `exprKindFromToken`. This function converts constants of type `TokenKind` into constants of type `ExprKind`. Take another look at the parser and remind yourself why this conversion is needed.
2. With a little experience, we would probably make several related changes at once. But let us deliberately play dumb for a moment — this gives us a nice opportunity to see a compile-time error as well. ;-) Extend `exprKindFromToken` to

   ```
   fn exprKindFromToken(token: Token): ExprKind
   {
       switch (token.kind) {
           case PLUS: return EXPR_ADD;
           case MINUS: return EXPR_SUB;
           case ASTERISK: return EXPR_MUL;
           case SLASH: return EXPR_DIV;
           case PERCENT: return EXPR_MOD;
           default:
               printf("token: %d (%s)\n", token.kind, tokenKindStr(token.kind));
               assert(0 && "internal error: exprKindFromToken");
               return 0;
       }
   }
   ```

© Michael C. Lehn | CC BY-SA 4.0

Now run `make` again. This time the compiler should stop with an error similar to

```
        case MINUS: return EXPR_SUB;
                           ^^^^^^^^^
parseexpr.abc:16.28-16.36: error: undefined identifier
```

Fair enough: we have never defined a constant called `EXPR_SUB`. **Where would be the appropriate place to define it? And while we are there, which other constants should we probably add as well?**

3. Extend `ExprKind` in `expr.hdr` accordingly:

```
enum ExprKind
{
    EXPR_ADD,
    EXPR_SUB,
    EXPR_MUL,
    EXPR_DIV,
    EXPR_MOD,

    EXPR_INTEGER,
};
```

Run make again. Everything should compile. Hooray — no errors! Well, at least none that the compiler can find for us. ;-)
Repeat our minimal test by running `./xtest_parseexpr` and entering `2 - 1`. We get another runtime error:

```
expr.abc:42: Assertion '0' failed.
```

Find the assertion in `expr.abc` and determine why our new expression reaches it.

4. The assertion is triggered in `printExprTree`. Its switch statement does not yet handle our new kinds of binary expressions. Extend it to

```
switch (expr->kind) {
    case EXPR_ADD: printf("+ "); break;
    case EXPR_SUB: printf("- "); break;
    case EXPR_MUL: printf("* "); break;
    case EXPR_DIV: printf("/ "); break;
    case EXPR_MOD: printf("mod "); break;
    default: assert(0);
}
```

Do not change anything else yet. Run `make` and try `2 - 1` again. This time the test should run without an error. So we are done. **Right?**

5. Let us try one slightly less trivial expression: `2 - 2 / 2`. `xtest_parseexpr` produces

```
[ - [2] ]
```

It has been a while since we worked with expression trees, so at first glance you may not even remember whether this is correct. There is no error message either. **Spoiler: it is wrong.**

The reason is not particularly obvious unless the implementation of `printExprTree` happens to be in your brain cache. Take another look at the function. The problem is this condition:

```
} else if (expr->kind >= EXPR_ADD && expr->kind <= EXPR_MUL) {
```

It is used to determine whether a node has two children — in other words, whether it is a binary-expression node. After adding new `ExprKind` constants, the condition is no longer correct. For now, use the quick fix

```
} else if (expr->kind >= EXPR_ADD && expr->kind <= EXPR_MOD) {
```

We will replace this with something better shortly. After a successful make, try several expressions containing all of our new operators and check that the resulting expression trees look correct.

6. Ask your neighbor whether they noticed that there is still another bug in our code. This one is particularly sneaky: it does not produce a visible wrong result at all. Due to the bug, we "only" leak memory. Find the corresponding place in our expression-tree code. You can either fix it analogously to the problem above — or leave it for the moment, because in the next step we will fix both problems properly.

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — Making `ExprKind` Less Error-Prone

Replace the declaration of `ExprKind` in `expr.hdr` by

```
enum ExprKind
{
    EXPR_BINARY,
    EXPR_ADD = EXPR_BINARY,
    EXPR_SUB,
    EXPR_MUL,
    EXPR_DIV,
    EXPR_MOD,
    EXPR_BINARY_END,

    EXPR_PRIMARY = EXPR_BINARY_END,
    EXPR_INTEGER,
    EXPR_PRIMARY_END,
};
```

and change the implementation of `printExprTree` in `expr.abc` to

```
fn printExprTree(expr: ->Expr)
{
    if (expr->kind >= EXPR_PRIMARY && expr->kind < EXPR_PRIMARY_END) {
        switch (expr->kind) {
            case EXPR_INTEGER:
                printf("[%s]", expr->primary.decimal);
                break;
            default: assert(0 && "node kind not handled");
        }
    } else if (expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END) {
        printf("[ ");
        switch (expr->kind) {
            case EXPR_ADD: printf("+ "); break;
            case EXPR_SUB: printf("- "); break;
            case EXPR_MUL: printf("* "); break;
            case EXPR_DIV: printf("/ "); break;
            case EXPR_MOD: printf("mod "); break;
            default: assert(0);
        }
        printExprTree(expr->binary.left);
        printf(" ");
        printExprTree(expr->binary.right);
        printf("] ");
    } else {
        assert(0 && "node kind not handled");
    }
}
```

In the following tasks, we explore some patterns that probably arose from long and repeated periods of suffering — and help us avoid such suffering in the future. ;-)

Tasks

1. Determine the numerical values of all constants in `ExprKind`. This is also a good opportunity to refresh how enum constants work.
2. The constants representing expression-tree nodes are now enclosed in two ranges: one for **binary nodes** and one for **primary nodes**. Convince yourself that for an `ExprKind kind`:
   - `kind >= EXPR_BINARY && kind < EXPR_BINARY_END` means that `kind` represents a binary node.
   - `kind >= EXPR_PRIMARY && kind < EXPR_PRIMARY_END` means that `kind` represents a primary node.

   Notice how `EXPR_ADD = EXPR_BINARY` and `EXPR_PRIMARY = EXPR_BINARY_END` make the boundaries work.
3. Look again at `printExprTree`. A node is first routed into the appropriate group by the range checks above and is then handled by a switch. Suppose we later add another kind of primary expression but forget to add the corresponding case to `printExprTree`. What happens? Why is the default: `assert(0 && "node kind not handled")` useful here?
4. There is still the memory leak mentioned at the end of the previous step. Adapt `releaseExpr` to use the same pattern for distinguishing primary and binary nodes. Make sure that all dynamically allocated parts of the expression tree are released correctly.

© Michael C. Lehn | CC BY-SA 4.0

Step 6 — Adding Keywords to the Lexer

Our lexer can already recognize identifiers, but it does not yet distinguish identifiers from keywords such as `if`, `while`, or `fn`. Let us change that.

You may, of course, anticipate the following changes and add all keywords at once. However, it can also be instructive to go through the process once or twice step by step and see how each required change reveals itself. Choose whichever approach you prefer.

Tasks

1. Let us start with the keywords `fn` and `if`. Extend `TokenKind` in `lexer.hdr` by adding the corresponding constants:

   ```
   enum TokenKind
   {
       // ... as before
       FN,
       IF,
   };
   ```

   Notice that, unlike ExprKind in the previous step, we do not introduce special ranges or groups here. There is a simple reason: so far, we have no need for them. Do not introduce a pattern unless there is a problem it actually solves.

2. Now adapt our getToken wrapper in `lexer.abc`:

   ```
   global kw_fn, kw_if: ->UStr;

   fn getToken(): TokenKind
   {
       static first: bool = true;
       if (first) {
           first = false;
           kw_fn = UStrCreate("fn");
           kw_if = UStrCreate("if");
       }
       getToken_();
       token.val = UStrCreate(internalVal);
       if (token.kind == IDENTIFIER) {
           if (token.val == kw_fn) {
               token.kind = FN;
           } else if (token.val == kw_if) {
               token.kind = IF;
           }
       }
       return token.kind;
   }
   ```

   There is quite a bit going on in these few lines. Make sure you understand the following points. Explain them to your neighbor — or discuss them with us:

   - Why is `getToken` a wrapper function again? Why did we introduce this wrapper?
   - Why can't we initialize the global variables directly?[2]
   - Why is first a static variable? What were the similarities and differences between local, global, and staticvariables again?
   - Why do we use this somewhat cumbersome `if`–`else if` chain instead of a `switch` after the `if (token.kind == IDENTIFIER)`?

3. Compile everything with make. Then run `./xtest_lexer` with input `iff` `if`. You should get

   ```
   1.1: IDENTIFIER (iff)
   1.5: ?? (if)
   ```

   Our keyword recognition seems to work — but there is obviously still something left to do to get rid of those ??. ;-)

Eventually, our lexer should recognize the following keywords:

Keyword	TokenKind	Keyword	TokenKind	Keyword	TokenKind	Keyword	TokenKind
fn	FN	global	GLOBAL	else	ELSE	do	DO
if	IF	local	LOCAL	while	WHILE	return	RETURN

[2] C++ does allow such initialization.

© Michael C. Lehn | CC BY-SA 4.0